

entities.Cliente.java

```
package entities;

public class Cliente {
    public String nome;
    public double saldo; // saldo nao pode ser publico, melhor usar uma função
    "get", depois faço isso
    public boolean especial;
    public String endereco;
}
```

entities.Banco.java

```
package entities;
import operations.*;
import java.util.ArrayList;

public class Banco {
    public String name;
    public int quantClientes=0;
    public Banco (String name){this.name = name;};
    public ArrayList<ContaCorrente> contas = new ArrayList<>();

    public void addContaCorrente(ContaCorrente conta){
        contas.add(conta);
        quantClientes += 1;
    }
    public String getNome(){return name;}

    public void imprimirContas(){
        System.out.println("----- CONTAS ATIVAS -----");
        for(ContaCorrente conta : contas){
            System.out.println(conta.cliente.nome); // melhorar isso depois;
        }
        System.out.println("-----");
    }
}
```

extras.DataHora.java

```
package extras;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class DataHora {
    // Obter a data e hora atuais
    public LocalDateTime dataHoraAtual = LocalDateTime.now();

    public String formatDataHora(){
        DateTimeFormatter formatador = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss");
        //String dataHoraFormatada = dataHoraAtual.format(formatador);
        // Formatar em um padrão específico (ex: dd/MM/yyyy HH:mm:ss)
        return dataHoraAtual.format(formatador);
    }
}
```

operations.ContaCorrente.java

```
package operations;

import java.util.ArrayList;
import java.util.Random;
import entities.*;
import extras.*;

public class ContaCorrente {
    public Cliente cliente;
    public int id;
    Banco banco;
    ArrayList<Transacoes> extrato;
    Random geradorID = new Random(); // melhorar isso depois;

    public ContaCorrente(Cliente cliente, Banco banco) {
        this.cliente = cliente;
        this.extrato = new ArrayList<Transacoes>();
        this.id = geradorID.nextInt(10000);
        banco.addContaCorrente(this);
    }

    public void mostrarInfo(){
        System.out.println("Usuario: " + cliente.nome);
        System.out.println("Endereco: " + cliente.endereco);
        if(cliente.especial){
            System.out.println("Conta: Especial");
        }else{
            System.out.println("Conta: Normal");
        }
        System.out.println("Saldo atual: " + cliente.saldo);
    }

    void emitirSaldo(){
        System.out.println("Usuario: " + cliente.nome);
        System.out.println("Saldo atual: " + cliente.saldo);
    }

    public void emitirExtrato(){
        System.out.println("--- EXTRATO " + cliente.nome + " ---");
        for(Transacoes t : extrato){
            t.formatarMensagem();
        }
    }

    boolean ehEspecial(){
        return cliente.especial;
    }

    void tipoUsuario(){
        if(ehEspecial()){
            System.out.println(cliente.nome + " eh usuario especial!");
        }else{
            System.out.println(cliente.nome + " eh usuario normal!");
        }
    }
}
```

```

public void depositar(double valor){
    cliente.saldo += valor;
    System.out.println("Valor depositado com sucesso!");
    System.out.println("Saldo atual: " + cliente.saldo);
    DataHora dataHoraAtual = new DataHora();
    Transacoes deposito = new Transacoes("depositou",this,valor, dataHoraAtual.formatDataHora());
    //String movimentacao = deposito.entradaDados1Usuario("depositou", id, cliente.saldo,
dataHoraAtual.formatDataHora());
    extrato.add(deposito);
}

public void sacar(double valor){
    if(valor > cliente.saldo && !ehEspecial()){
        System.out.println("Nao foi possivel sacar. Saldo insuficiente!");
    }else{
        cliente.saldo -= valor;
        System.out.println("Valor retirado com sucesso!");
        System.out.println("Saldo restante: " + cliente.saldo);
        DataHora dataHoraAtual = new DataHora();
        Transacoes saque = new Transacoes("sacou",this,valor, dataHoraAtual.formatDataHora());
        extrato.add(saque);
    }
}

public void transferir(double valor, ContaCorrente conta2){
    if(valor > cliente.saldo && !ehEspecial()){
        System.out.println("Nao foi possivel transferir. Saldo insuficiente!");
    }else{
        cliente.saldo -= valor;
        conta2.cliente.saldo += valor;
        System.out.println(valor + " transferido com sucesso para " + conta2.cliente.nome);
        System.out.println("Saldo restante: " + cliente.saldo);
        DataHora dataHoraAtual = new DataHora();
        Transacoes transferencia = new Transacoes("transferiu",this,valor, dataHoraAtual.formatDataHora());
        Transacoes transferido = new Transacoes("recebeu",conta2,valor, dataHoraAtual.formatDataHora());
        extrato.add(transferencia);
        conta2.extrato.add(transferido);
    }
}
}

```

operations.Transacoes.java

```
package operations;
import entities.*;

public class Transacoes {
    String acao;
    ContaCorrente contaCorrente;
    double valor;
    String data;

    public Transacoes(String acao, ContaCorrente contaCorrente, double valor,
String data) {
        this.acao = acao;
        this.contaCorrente = contaCorrente;
        this.valor = valor;
        this.data = data;
    }

    public void formatarMensagem(){
        System.out.println("Usuario " + contaCorrente.id + " " + acao + " " + valor + "
| " + data);
    }
}
```

interfaceUsuario.Aplicacao.java

```
package interfaceUsuario;
import java.util.*;
import entities.*;
import operations.ContaCorrente;

public class Aplicacao {
    private List<Banco> bancos = new ArrayList<>();
    Scanner scanner = new Scanner(System.in);

    public void execucao(){
        int opcao;
        do {
            System.out.println("\nMenu:");
            System.out.println("1. Criar Banco");
            System.out.println("2. Buscar Banco");
            System.out.println("3. Listar Bancos");
            System.out.println("4. Cadastrar Usuario em Banco");
            System.out.println("5. Buscar Usuario de Banco");
            System.out.println("6. Extrato de Usuario");
            System.out.println("7. Simular Saque");
            System.out.println("8. Simular Deposito");
            System.out.println("9. Simular Transferencia");
            System.out.println("10. Listar Clientes Banco");
            //System.out.println("0. Sair");
            System.out.println("0. Sair");
            System.out.print("Opção: ");
            opcao = scanner.nextInt();
            scanner.nextLine();

            switch (opcao) {
                case 1 -> this.criarBanco();
                case 2 -> buscarBanco();
                case 3 -> listarBancos();
                case 4 -> cadastrarCliente();
                case 5 -> infoCliente();
                case 6 -> mostrarExtratoUsuario();
                case 7 -> simulacaoSaque();
                case 8 -> simulacaoDeposito();
                case 9 -> simulacaoTransferencia();
                case 10 -> listarContasBanco();
            }
        } while (opcao != 0);
    }

    private void criarBanco(){
        System.out.println("Nome do Banco:");
        String nome = scanner.nextLine();
        bancos.add(new Banco(nome));
        System.out.println("Banco " + nome + " criado!");
    }

    private Banco buscarBanco(){
        System.out.println("Nome do Banco:");
        String nome = scanner.nextLine();
        for (Banco banco : bancos) {
            if(banco.name.equals(nome)){
                return banco;
            }
        }
        System.out.println("Banco " + nome + " nao cadastrado!");
        return null;
    }
}
```

```

private void listarBancos(){
    System.out.println("--- BANCOS CADASTRADOS ---");
    int i=0;
    for (Banco banco : bancos){
        System.out.println((i+1) + " - " + banco.name);
        i++;
    }
}

private void cadastrarCliente(){
    Banco banco = buscarBanco();
    if(banco == null){
        System.out.println("Banco nao existe!");
        return;
    }
    Cliente novoCliente = new Cliente(); // ver isso depois, write-only object;
    System.out.println("Nome: ");
    novoCliente.nome = scanner.nextLine();
    System.out.println("Endereco: ");
    novoCliente.endereco = scanner.nextLine();
    System.out.println("Saldo: ");
    novoCliente.saldo = scanner.nextDouble();
    System.out.println("Cliente Especial? (True / False): ");
    novoCliente.especial = scanner.nextBoolean();
    ContaCorrente novaCC = new ContaCorrente(novoCliente, banco);
}

private ContaCorrente buscarCliente(){ // busca pela contaCorrente do usuario;
    Banco banco = buscarBanco();
    if(banco == null){
        System.out.println("Banco nao existe!");
        return null;
    }
    System.out.println("Nome: ");
    String nomeCliente = scanner.nextLine();
    for (ContaCorrente conta : banco.contas) {
        if(conta.cliente.nome.equals(nomeCliente)){
            return conta;
        }
    }
    System.out.println("Cliente " + nomeCliente + " nao cadastrado!");
    return null;
}

private void infoCliente(){
    ContaCorrente contaCliente = buscarCliente();
    if(contaCliente == null){
        System.out.println("Cliente nao existe!");
        return;
    }
    contaCliente.mostrarInfo();
}

private void mostrarExtratoUsuario(){
    ContaCorrente cliente = buscarCliente();
    if(cliente == null){
        System.out.println("Operacao falhou!");
        return;
    }
    cliente.emitirExtrato();
}

```

```

private void simulacaoSaque(){
    ContaCorrente contaCliente = buscarCliente();
    if(contaCliente == null){
        System.out.println("Operacao falhou!");
        return;
    }
    System.out.println("Valor a ser resgatado: ");
    double valor = scanner.nextDouble();
    contaCliente.sacar(valor);
}

private void simulacaoDeposito(){
    ContaCorrente contaCliente = buscarCliente();
    if(contaCliente == null){
        System.out.println("Operacao falhou!");
        return;
    }
    System.out.println("Valor a ser depositado: ");
    double valor = scanner.nextDouble();
    contaCliente.depositar(valor);
}

private void simulacaoTransferencia(){
    System.out.println("Dados conta 1 (conta que vai transferir): ");
    ContaCorrente contaCliente1 = buscarCliente();
    if(contaCliente1 == null){
        System.out.println("Operacao falhou!");
        return;
    }
    System.out.println("Dados conta 2 (conta que vai receber): ");
    ContaCorrente contaCliente2 = buscarCliente();
    if(contaCliente2 == null){
        System.out.println("Operacao falhou!");
        return;
    }

    System.out.println("Valor a ser transferido: ");
    double valor = scanner.nextDouble();
    contaCliente1.transferir(valor, contaCliente2);
}

private void listarContasBanco(){
    Banco banco = buscarBanco();
    if(banco == null){
        System.out.println("Banco nao existe!");
        return;
    }
    int i=0;
    for(ContaCorrente conta : banco.contas){
        System.out.println("Conta "+ (i+1) + ": " + conta.id + " - " + conta.cliente.nome);
        i++;
    }
}
}

```


Main.java

```
import entities.*;
import operations.*;
import extras.*;
import interfaceUsuario.*;

public class Main{
    public static void main(String[] args){
        Aplicacao app = new Aplicacao();
        app.execucao();
    }
}
```